

Organisation des données

: PostgreSQL + MongoDB

Quelles bases de données pour stocker 2 millions de bougies et un flux temps réel, pourquoi celles-là plutôt que les autres, et comment les deux se relient.

Dépôt DataScientest-Studio/jul26_bde_crypto

Échéance 4 septembre 2026

2 075 570 LIGNES À STOCKER	132 Mo VOLUME ACTUEL	2 BASES RETENUES	3 MODÈLES, UN PAR PROFIL
--------------------------------------	--------------------------------	----------------------------	---------------------------------------

01 Le chiffre qui cadre la décision

Avant de comparer quoi que ce soit, il faut savoir ce qu'on stocke. À l'issue de l'étape 1 : **2 075 570 lignes, 132 Mo**, sur 7 pas de temps et 5 paires.

C'est **petit**. PostgreSQL commence à ralentir vers **50 millions de lignes**, quand ses index ne tiennent plus en mémoire. Nous sommes à 4 % de ce seuil. Même en collectant les cinq paires à la minute pendant dix ans, nous resterions en dessous.

CE QUE ÇA ÉLIMINE

Tout l'argumentaire « il faut une base conçue pour le gros volume » ne s'applique pas ici. Le critère de choix n'est pas la performance brute — **c'est la forme des données**. Nous avons dimensionné avant de choisir, et non l'inverse.

02 Les solutions évaluées

SOLUTION	POINTS FORTS	POINTS FAIBLES	VERDICT
PostgreSQL	Transactions ACID, jointures, contraintes d'intégrité. Index BRIN très efficaces sur des données ordonnées dans le temps. Déjà maîtrisé par l'équipe. Conteneurisation immédiate	Pas de fonctions temporelles avancées en natif	RETENU
MongoDB	Schéma libre : absorbe des documents de formes différentes sans migration. Idéal pour du JSON d'API. Déjà maîtrisé par l'équipe	Jointures coûteuses. Les collections <i>time series</i> sont très contraintes : pas de <code>change streams</code> , champs non modifiables après écriture, index limités	RETENU
PostgreSQL + TimescaleDB	Hypertables, compression jusqu'à 10:1, agrégats continus (dériver le 1h depuis le 1m automatiquement), politiques de rétention intégrées	La gestion des <i>chunks</i> coûte plus qu'elle ne rapporte sous ~10 M de lignes. Licence TSL, pas strictement open source	PLUS TARD
Elasticsearch	Recherche plein texte, agrégations rapides, très bon outillage de visualisation	Conçu pour la recherche, pas pour du stockage transactionnel. Gourmand en mémoire, réglage lourd, pas de garantie ACID	ÉCARTÉ
InfluxDB / ClickHouse	ClickHouse agrège 6 à 7× plus vite que TimescaleDB. Ingestion de plusieurs millions de points par seconde	Dimensionnés pour des milliards de lignes. Pas d'ACID. Aucun des deux n'a été vu en formation	ÉCARTÉ
Snowflake / BigQuery	Élasticité, aucune administration, montée en charge transparente	Payants, cloud obligatoire. Incompatibles avec l'étape 4 , qui impose de conteneuriser tout le projet pour qu'il tourne sur n'importe quelle machine	ÉCARTÉ

03 Le critère de répartition

Le réflexe serait de découper « SQL pour l'historique, NoSQL pour le temps réel ». Nous avons écarté ce découpage : il sépare par **provenance**, or nous avons démontré à l'étape 1 qu'une bougie clôturée arrivée par WebSocket est *exactement* la même chose qu'une bougie arrivée par l'API REST. Les deux sources convergent déjà vers un schéma unique.

Nous découpons donc par **forme de la donnée** :

Structure stable et connue à l'avance → relationnel.
Structure variable ou imbriquée → document.

Ce n'est pas une règle théorique : nos propres données l'imposent. Les métadonnées renvoyées par `exchangeInfo` contiennent une liste de filtres où **chaque type de filtre a des clés différentes** :

PRICE_FILTER	→	filterType, minPrice, maxPrice, tickSize
LOT_SIZE	→	filterType, minQty, maxQty, stepSize
NOTIONAL	→	filterType, minNotional, applyToMarket, avgPriceMins

En relationnel, il faudrait une table par type de filtre, ou une table clé-valeur générique — l'anti-pattern EAV, qui rend toute requête illisible. En document, c'est un sous-document et rien d'autre. Même problème pour le carnet d'ordres, dont les offres arrivent en tableaux de tableaux : `[["79324.00", "3.13"], ...]`.

À l'inverse, une bougie a **treize colonnes, toujours les mêmes**, avec une clé métier `(symbol, interval, open_time)` déjà validée sans doublon à l'étape 1. C'est un cas d'école relationnel.

PostgreSQL

COUCHE EXPLOITABLE

- Les bougies nettoyées, 13 colonnes fixes
- Le référentiel des paires et des pas de temps
- Le journal des collectes et leur qualité
- C'est cette couche que le modèle de ML consommera à l'étape 3

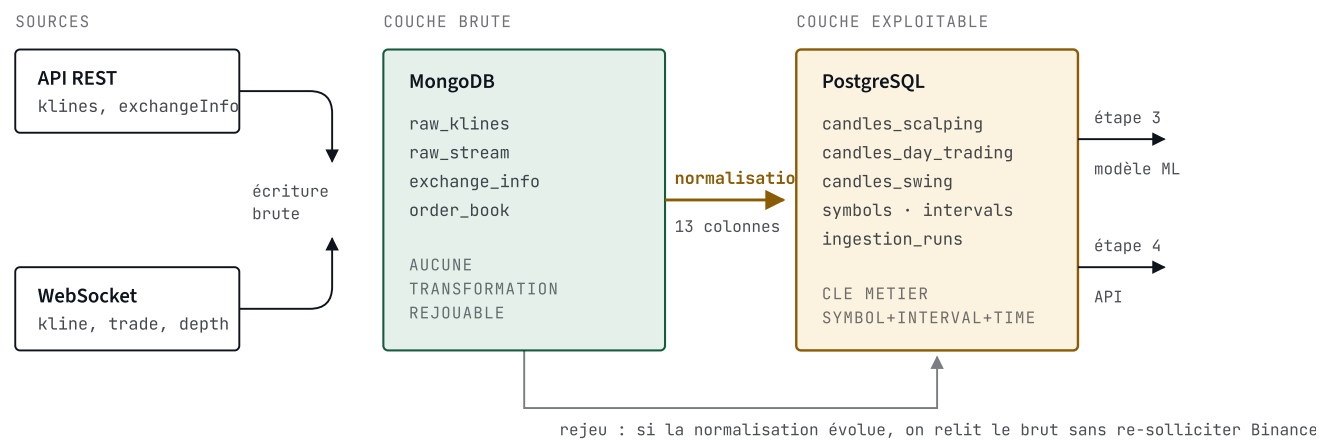
MongoDB

COUCHE BRUTE

- Les réponses de l'API telles quelles
- `exchangeInfo` et ses filtres hétérogènes
- Les messages WebSocket de tous types — `kline`, `trade`, `bookTicker` ont des formes différentes
- Les instantanés du carnet d'ordres

Cette architecture n'est pas nouvelle : **c'est déjà la vôtre**. Le dossier `data/raw/` devient MongoDB, `data/processed/` devient PostgreSQL. Nous transposons en base une séparation qui a déjà été justifiée et éprouvée.

04 Le flux de bout en bout



Tout entre par MongoDB sous sa forme d'origine, puis est normalisé vers PostgreSQL. La flèche du bas est ce qui justifie de garder les deux : le brut permet de rejouer une transformation corrigée sans redemander six ans de données à Binance.

À 5 bougies clôturées par minute, faire transiter le temps réel par MongoDB ne coûte rien. Si la latence devenait critique, les bougies closes pourraient aller directement en PostgreSQL — le schéma pivot rend les deux chemins équivalents.

05 Un modèle par profil, sans duplication

Chaque profil de trading a sa propre table de faits. Mais deux pas de temps sont réclamés par deux profils à la fois : 15m par le scalping et le day trading, 4h par le day trading et le swing. Les stocker dans les deux tables dupliquerait les périodes qui se recouvrent.

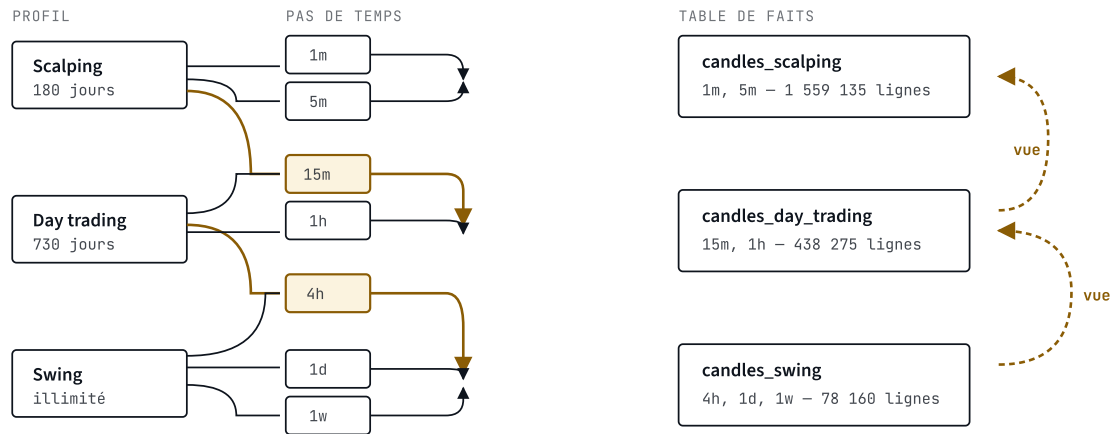
La règle appliquée est celle qui existe déjà dans le collecteur : la profondeur la plus longue l'emporte. Chaque pas de temps est stocké une seule fois, dans la table du profil qui le conserve le plus longtemps. Les autres profils y accèdent par une vue.

STOCKER N'EST PAS UTILISER

Chaque profil utilise toujours ses trois pas de temps. Ce qui varie, c'est l'endroit où ils sont rangés. Un profil qui n'en stocke que deux lit le troisième dans la table voisine, sans le dupliquer.

PROFIL	PAS DE TEMPS UTILISÉS	STOCKÉS DANS SA TABLE	LUS PAR VUE
Scalping	1m 5m 15m	1m 5m	15m ← candles_day_trading
Day trading	15m 1h 4h	15m 1h	4h ← candles_swing
Swing	4h 1d 1w	4h 1d 1w	—

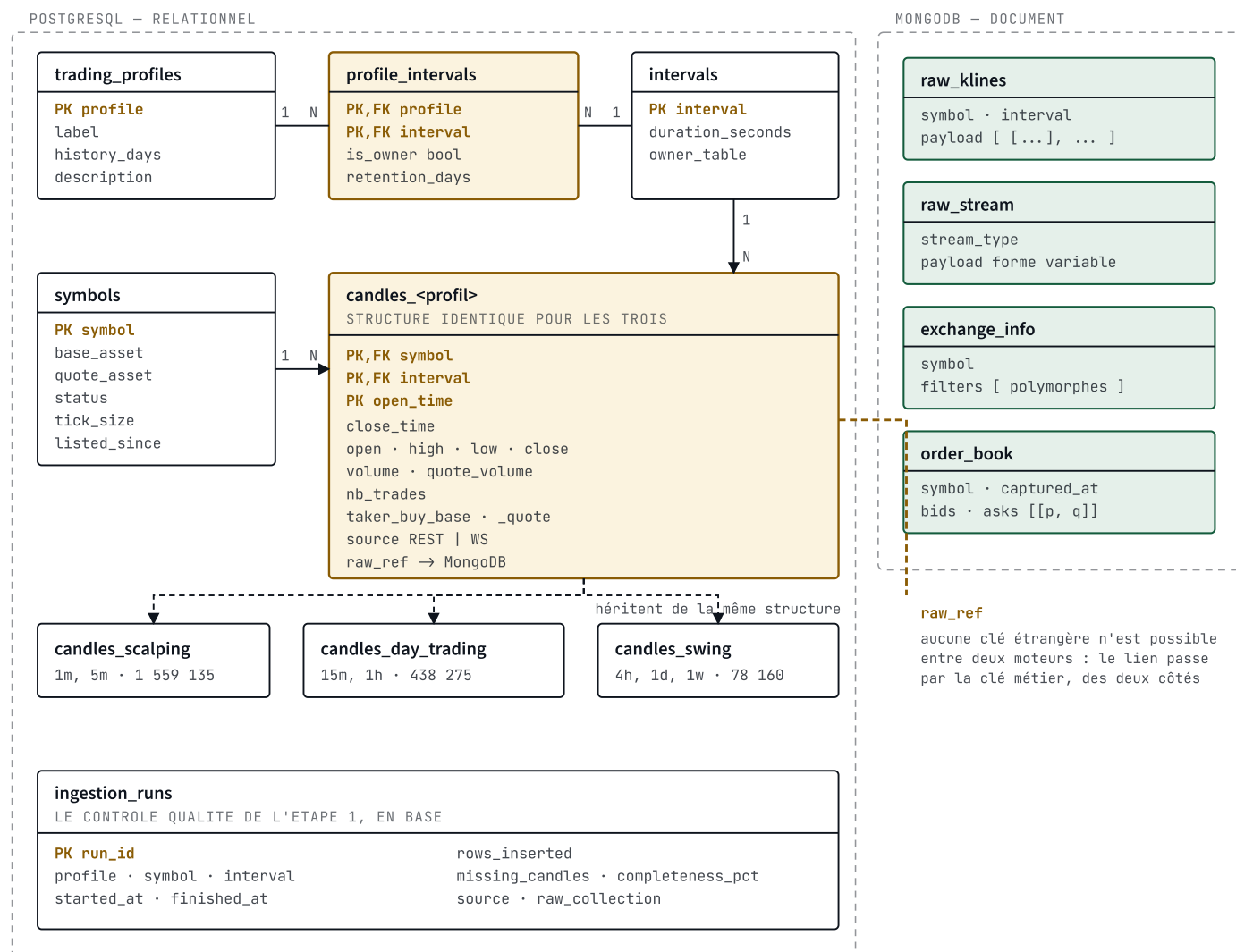
Concrètement, v_scalping restitue bien les trois pas de temps : le modèle de l'étape 3 interroge une vue et ignore complètement dans quelle table physique chaque bougie réside.



Les deux pas de temps en ambre sont partagés entre deux profils. Ils sont stockés une seule fois, dans la table dont la rétention est la plus longue, et remontés à l'autre profil par une vue en pointillés. Chaque profil conserve donc l'usage de ses trois pas de temps. Total stocké : 2 075 570 lignes, exactement le nombre collecté, sans un doublon.

Ce découpage a un second bénéfice, plus important que l'économie d'espace : **la rétention devient une opération de table entière**. Purger le scalping au-delà de 180 jours, c'est supprimer d'anciennes partitions — pas exécuter un `DELETE` massif qui fragmenterait la table et bloquerait les écritures.

06 Le modèle de données



Le référentiel en haut, les faits au centre, le journal des collectes en bas. À droite, les quatre collections MongoDB — reliées non par une clé étrangère, impossible entre deux moteurs, mais par la clé métier `symbol + interval + open_time` que chaque document porte.

Deux choix à défendre

L'indicateur `is_owner` dans `profile_intervals`. C'est lui qui encode la règle de la section 05 dans le modèle lui-même, plutôt que dans du code. Une ligne `(scalping, 15m, is_owner = false)` dit : « le scalping utilise le 15m mais ne le stocke pas ». Le modèle est ainsi auto-documenté, et changer la répartition ne demande pas de toucher au schéma.

La table `ingestion_runs`. C'est le `rapport_qualite.json` de l'étape 1, transposé en base. Chaque collecte y laisse une trace : combien de lignes, quelle complétude, quels trous. Cela répond à une exigence de l'étape 4 — mesurer la dérive des données — et à celle de l'étape 5, monitorer la production. Sans cette table, il n'y aurait aucun moyen de savoir *quand* une donnée est entrée ni si la collecte s'est bien passée.

07 Comment les deux bases sont reliées

Aucun moteur ne peut imposer une clé étrangère vers l'autre. Le lien repose donc sur la **clé métier**, présente des deux côtés :

```
-- PostgreSQL : une bougie et sa trace vers le brut
symbol = 'BTCUSDT', interval = '1h', open_time = '2026-08-28T12:00:00Z'
raw_ref = 'raw_klines/652f1a...'

// MongoDB : le document d'origine porte les memes identifiants
{ _id: ObjectId("652f1a..."),
  symbol: "BTCUSDT", interval: "1h",
  fetched_at: ISODate("2026-08-28T14:43:59Z"),
  payload: [ [1787918400000, "79604.12", ... ] ] }
```

Depuis n'importe quelle ligne de PostgreSQL, on peut donc remonter à la réponse exacte de Binance qui l'a produite. C'est la **traçabilité** : si une valeur paraît suspecte à l'étape 3, on vérifie l'original au lieu de spéculer.

08 Ce que nous avons écarté, et à quelle condition nous reviendrions dessus

SOLUTION	RAISON DE L'ÉCARTER	CE QUI NOUS FERAIT CHANGER D'AVIS
TimescaleDB	Sa gestion de <i>chunks</i> coûte plus qu'elle ne rapporte sous 10 M de lignes	Dépasser 50 M de lignes. C'est une extension PostgreSQL : la bascule est une commande SQL, pas une migration
Elasticsearch	Moteur de recherche, pas de stockage transactionnel	Un besoin de recherche plein texte — par exemple si on ajoutait des actualités ou du sentiment de marché à l'étape 3
ClickHouse	Conçu pour des milliards de lignes, pas d'ACID	Passer à des données tick par tick : le flux <code>trade</code> seul produit 136 messages par seconde sur BTCUSDT
Snowflake / BigQuery	Cloud payant, incompatible avec la conteneurisation exigée à l'étape 4	Rien dans le cadre de ce projet
Redis	Non retenu pour ne pas ajouter un troisième moteur	Si l'API de l'étape 4 devait servir le dernier prix connu à haute fréquence

LE RAISONNEMENT EN UNE PHRASE

Nous n'avons pas choisi les outils les plus puissants, mais **ceux dont la forme correspond à celle de nos données**, à un volume que nous avons mesuré avant de décider. Chaque solution écartée l'est pour une raison chiffrée, avec le seuil à partir duquel elle redeviendrait pertinente.

09 La suite

Les deux bases tourneront dans un `docker-compose` — PostgreSQL et MongoDB — ce qui prend de l'avance sur l'étape 4, qui impose de conteneuriser l'ensemble du projet.

Le pipeline d'injection réutilise ce qui existe : `src/preprocessing.py` produit déjà exactement le schéma des tables de faits. Il ne manque qu'une couche d'écriture entre le nettoyage et la base.

Points à valider

1. **L'horizon de prédiction**, toujours ouvert depuis l'étape 1. Il désignera le profil principal, et donc la table sur laquelle l'étape 3 travaillera en priorité.
2. **La politique de rétention**. Purge-t-on réellement le scalping au-delà de 180 jours, ou conserve-t-on tout tant que le volume reste modeste ?
3. **La granularité de la couche brute**. Conserve-t-on l'intégralité des messages WebSocket, y compris les 96 % de bougies non clôturées, ou seulement ce qui est retenu ?